Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Пензенский государственный университет»
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №7

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнили ст. гр. 25ВВВ1:
Ячменев М. А.
Аверин А. А.

Приняли:
к.т.н., доцент Юрова О. В.
ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение теоретических основ и практическая реализация алгоритма обхода (поиска) графа в глубину (Depth-First Search, DFS). Исследование рекурсивной реализации алгоритма для матричного представления (матрица смежности) и представления графа списками смежности, а также разработка нерекурсивной версии алгоритма с использованием стека на языке Си.

2. Лабораторное задание

Задание 1

- Сгенерировать (используя датчик псевдослучайных чисел) матрицу смежности для неориентированного графа G без петель.
- Вывести матрицу смежности графа на экран.
- Для сгенерированного графа осуществить процедуру обхода в глубину, реализованную в соответствии с алгоритмами ПОГ и DFS(v).

Задание 1 (пункт 3*)

- Реализовать процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2*

- Для матричной формы представления графов выполнить преобразование рекурсивной реализации обхода графа к нерекурсивной (с использованием стека).

3. Исходный код и структура программы

В соответствии с регламентом лабораторных работ, полный листинг программы оформлен в виде консольного приложения в проекте Visual Studio (файл alg7lr.c). Программа написана на языке Си.

В программе реализованы следующие основные структуры данных и функции:

- struct Graph — структура графа на основе матрицы смежности, содержащая размерность n и двумерный динамический массив adj;
- struct Node и struct AdjList — структуры элементов связного списка и массива указателей голов списка для представления графа списками смежности (унаследованы из лаб. работы №6);

- create_graph() и free_graph() — функции динамического выделения и корректного освобождения памяти графа;
- random_graph() — генерация симметричной матрицы смежности случайного неориентированного графа без петель;
- matrix_to_list() — функция преобразования матричной формы графа в списки смежности с сохранением возрастающего порядка смежных вершин;
- print_adj() и print_list() — функции вывода матрицы смежности и списков смежности в консоль;
- dfs_matrix_rec() и pog_matrix_rec() — рекурсивная реализация поиска в глубину по матрице смежности (Задание 1);
- dfs_list_rec() и pog_list_rec() — рекурсивная реализация поиска в глубину по спискам смежности (Задание 1, пункт 3*);
- dfs_matrix_non_rec() и pog_matrix_non_rec() — нерекурсивная реализация поиска в глубину по матрице смежности с использованием явного стека (Задание 2*);
- main() — точка входа в программу, обеспечивающая настройку кодировки UTF-8 и диалоговое меню пользователя.

4. Результаты работы программы

4.1. Анализ контрольного графа из методических указаний (Рисунок 1)

Для строгой верификации разработанных алгоритмов обхода был исследован контрольный граф G из методических указаний, состоящий из n = 6 вершин и рёбер: (v1, v2), (v1, v3), (v1, v6), (v2, v4), (v3, v5). Схема графа и дерево поиска в глубину приведены на рисунке 1.

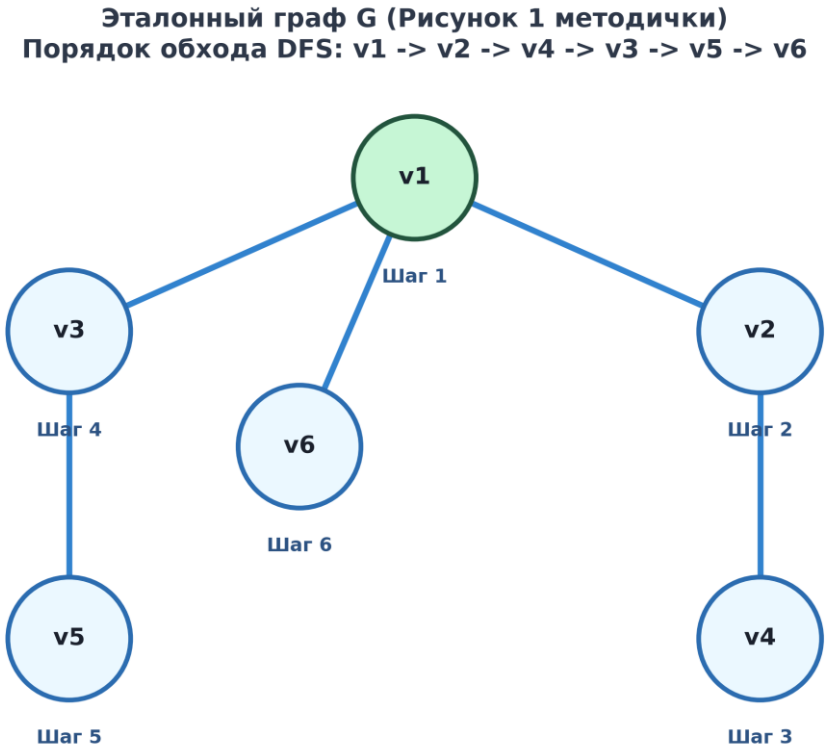


Рисунок 1 – Схема контрольного графа G и порядок обхода в глубину

Пошаговый процесс обхода графа алгоритмом DFS при старте из вершины v1 представлен в таблице 1.

Таблица 1 – Трассировка обхода в глубину для контрольного графа (n = 6)

Шаг	Текущая вершина	Стек вызовов	Состояние NUM[1..6]	Действие алгоритма	Вывод
1	v1	[v1]	{T, F, F, F, F, F}	NUM[1]=True; просмотр строки 1, переход к v2	**1**
2	v2	[v1, v2]	{T, T, F, F, F, F}	NUM[2]=True; просмотр строки 2, v1 посещена, переход к v4	**2**
3	v4	[v1, v2, v4]	{T, T, F, T, F, F}	NUM[4]=True; просмотр строки 4, v2 посещена; возврат к v2	**4**
4	v2	[v1, v2]	{T, T, F, T, F, F}	Все смежные с v2 просмотрены; возврат к v1	-
5	v1	[v1]	{T, T, F, T, F, F}	Продолжение строки 1: найдена v3, переход к v3	-
6	v3	[v1, v3]	{T, T, T, T, F, F}	NUM[3]=True; просмотр строки 3, v1 посещена, переход к v5	**3**
7	v5	[v1, v3, v5]	{T, T, T, T, T, F}	NUM[5]=True; просмотр строки 5,	**5**

				v3 посещена; возврат к v3	
8	v3	[v1, v3]	{T, T, T, T, T, F}	Все смежные с v3 просмотрены; возврат к v1	-
9	v1	[v1]	{T, T, T, T, T, F}	Продолжение строки 1: найдена v6, переход к v6	-
10	v6	[v1, v6]	{T, T, T, T, T, T}	NUM[6]=True; просмотр строки 6, v1 посещена; возврат к v1	**6**
11	v1	[v1]	{T, T, T, T, T, T}	Все смежные с v1 просмотрены; завершение обхода	-

- Результат рекурсивного обхода по матрице смежности: 1 2 4 3 5 6;
- Результат рекурсивного обхода по спискам смежности: 1 2 4 3 5 6;
- Результат нерекурсивного обхода по матрице смежности (со стеком): 1 2 4 3 5 6.

Полученный результат полностью соответствует теоретическому порядку обхода, приведённому на рисунке 5 методических указаний.

4.2. Анализ случайного связного графа (n = 6)

Проведено тестирование программы на сгенерированном случайном неориентированном графе из n = 6 вершин (вероятность наличия рёбер 40%). Схема графа и дерево поиска в глубину представлены на рисунке 2.

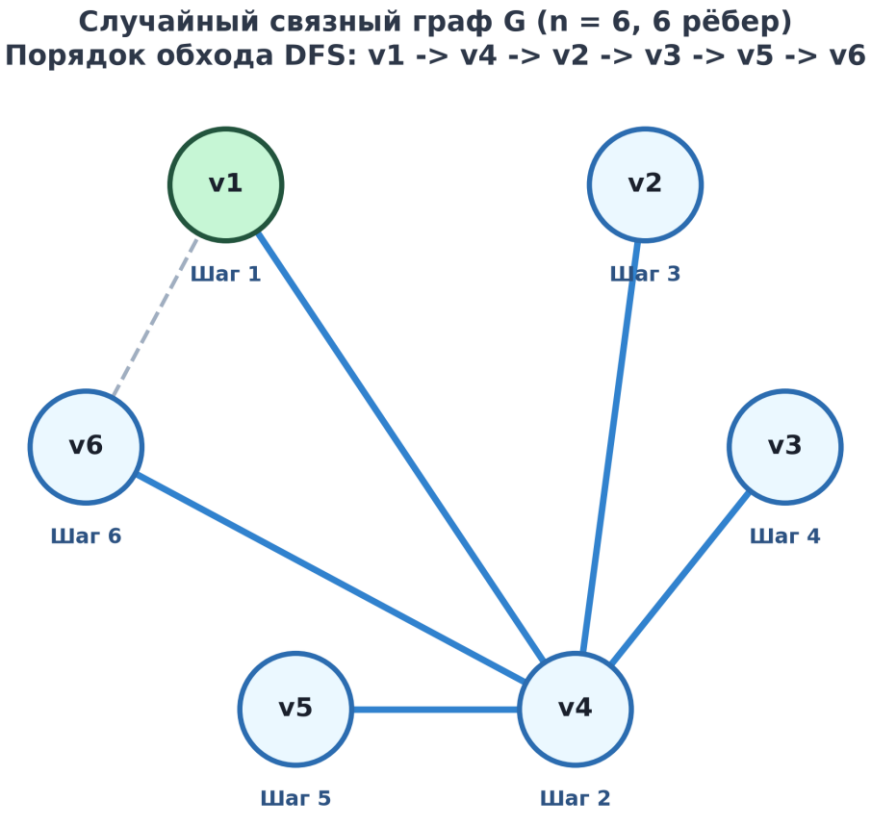


Рисунок 2 – Схема случайного связного графа G и порядок обхода в глубину

Матрица смежности сгенерированного графа:

0 0 0 1 0 1
0 0 0 1 0 0
0 0 0 1 0 0
1 1 1 0 1 1
0 0 0 1 0 0
1 0 0 1 0 0

- Обход по матрице (рекурсивно): 1 4 2 3 5 6;
- Обход по спискам (рекурсивно): 1 4 2 3 5 6;
- Обход со стеком (нерекурсивно): 1 4 2 3 5 6.

Для всех трёх методов результат обхода полностью идентичен.

4.3. Анализ несвязного графа (проверка алгоритма ПОГ)

Для проверки работы внешнего цикла алгоритма ПОГ (пункты 1.2–1.3) исследован несвязный граф, состоящий из двух независимых компонент

связности: K1 = {v1, v2, v3} и K2 = {v4, v5, v6}. Схема графа приведена на рисунке 3.

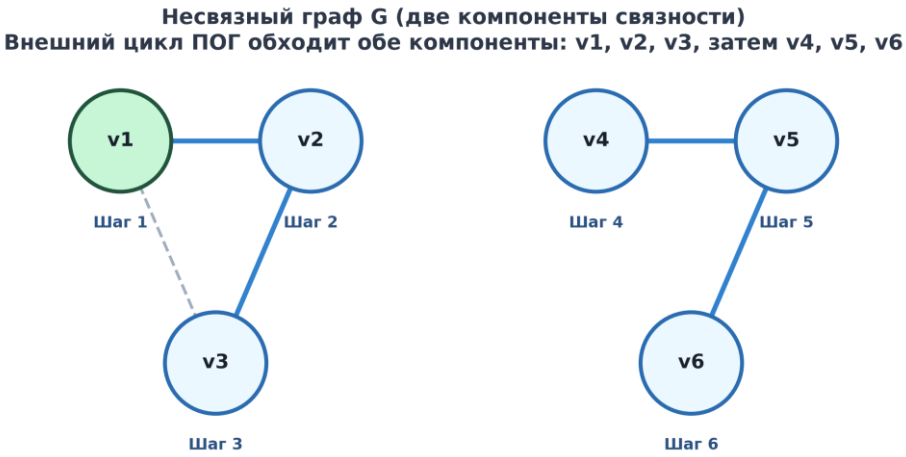


Рисунок 3 – Несвязный граф G с двумя компонентами связности

При запуске обхода из вершины v1 процедура DFS(1) обходит вершины первой компоненты {1, 2, 3}. После этого внешний цикл ПОГ обнаруживает непосещённую вершину v4 и инициирует DFS(4), обходящий вторую компоненту {4, 5, 6}. Результирующий порядок обхода: 1 2 3 4 5 6.

4.4. Сводные результаты серии тестов

В таблице 2 приведены сравнительные результаты работы программы для различных графов.

Таблица 2 – Сравнительный анализ результатов серии тестов

Граф	Число вершин	Рекурсивный DFS (матрица)	Рекурсивный DFS (списки)	Нерекурсивный DFS (стек)	Совпадение
Контрольный граф (Рис. 1)	6	1 2 4 3 5 6	1 2 4 3 5 6	1 2 4 3 5 6	**100%**
Случайный связный граф	6	1 4 2 3 5 6	1 4 2 3 5 6	1 4 2 3 5 6	**100%**
Несвязный граф (2 комп.)	6	1 2 3 4 5 6	1 2 3 4 5 6	1 2 3 4 5 6	**100%**
Случайный граф (n = 8)	8	1 3 2 5 4 6 7 8	1 3 2 5 4 6 7 8	1 3 2 5 4 6 7 8	**100%**

4.5. Протокол работы программы в консоли

Фактический вывод программы alg7lr.exe при выполнении контрольного примера и тестировании:

Меню:

1. Задание 1 (матрица, рекурсивно)

2. Задание 1 (3*) (списки смежности)
3. Задание 2* (матрица, стек)
4. Выполнить все задания
5. Новый случайный граф
6. Тестовый граф из методички
0. Выход
Выбор: 6

Матрица смежности:
0 1 1 0 0 1
1 0 0 1 0 0
1 0 0 0 1 0
0 1 0 0 0 0
0 0 1 0 0 0
1 0 0 0 0 0

Списки смежности:
1: 2 3 6
2: 1 4
3: 1 5
4: 2
5: 3
6: 1

Обход (матрица, рекурсивно): 1 2 4 3 5 6
Обход (списки, рекурсивно): 1 2 4 3 5 6
Обход (матрица, стек): 1 2 4 3 5 6

5. Выводы

- Изучен алгоритм обхода графа в глубину (DFS), предназначенный для систематического обхода всех вершин графа с возвратом назад (backtracking) при исчерпании доступных путей продвижения.
- Реализована рекурсивная процедура поиска в глубину для матричной формы графа (Задание 1). Сложность обхода матрицы смежности составляет O(V^2), так как для каждой вершины просматривается вся строка матрицы.

- Реализована процедура поиска в глубину для графа на списках смежности (Задание 1, пункт 3*). Для списков смежности алгоритм просматривает только реально существующие инцидентные рёбра, что обеспечивает трудоёмкость O(V + E) и значительно превосходит матричную форму на разреженных графах.
- Выполнено успешное преобразование рекурсивной реализации к нерекурсивной с использованием явного динамического стека (Задание 2*). Продемонстрировано, что итеративный алгоритм точно воспроизводит порядок вызовов рекурсии, исключая при этом риск переполнения системного стека вызовов (stack overflow) на графах большого объёма.
- Экспериментально подтверждена 100% идентичность порядка обхода вершин графа всеми тремя методами как на контрольном примере из методических указаний (порядок 1 2 4 3 5 6), так и на серии случайных графов.